Latest updates: https://dl.acm.org/doi/10.1145/3701551.3703546

RESEARCH-ARTICLE

Large Language Model Simulator for Cold-Start Recommendation

FEIRAN HUANG, Jinan University, Guangzhou, Guangdong, China

YUANCHEN BEI, Zhejiang University, Hangzhou, Zhejiang, China

ZHENGHANG YANG, Jinan University, Guangzhou, Guangdong, China

JUNYI JIANG, Jinan University, Guangzhou, Guangdong, China

HAO CHEN, City University of Macau, Taipa, Macao

QIJIE SHEN, Alibaba Group Holding Limited, Hangzhou, Zhejiang, China

View all

Open Access Support provided by:

Jinan University

University of Illinois at Chicago

City University of Macau

Mohamed Bin Zayed University of Artificial Intelligence

Central South University

Alibaba Group Holding Limited

View all

PDF Download
3701551.3703546.pdf
17 December 2025
Total Citations: 28
Total Downloads: 1551

Published: 10 March 2025

Citation in BibTeX format

WSDM '25: The Eighteenth ACM International Conference on Web Search and Data Mining
March 10 - 14, 2025
Hannover, Germany

Conference Sponsors:
SIGKDD
SIGMOD
SIGIR
SIGWEB

WSDM '25: Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (March 2025)
https://doi.org/10.1145/3701551.3703546
ISBN: 9798400713293

Large Language Model Simulator for Cold-Start Recommendation

Feiran Huang
huangfr@jnu.edu.cn
Jinan University
Guangzhou, China

Yuanchen Bei
Zhejiang University
Hangzhou, China
yuanchenbei@zju.edu.cn

Zhenghang Yang
Jinan University
Guangzhou, China
yangzhenghang@stu2022.jnu.edu.cn

Junyi Jiang
Jinan University
Guangzhou, China
jjy0116@stu2022.jnu.edu.cn

Hao Chen*
City University of Macau
Macao, China
sundaychenhao@gmail.com

Qijie Shen
Alibaba Group
Hangzhou, China
qjshenxdu@gmail.com

Senzhang Wang
Central South University
Changsha, China
szwang@csu.edu.cn

Fakhri Karray
Mohamed Bin Zayed University of Artificial Intelligence
Abu Dhabi, UAE
fakhri.karray@mbzuai.ac.ae

Philip S. Yu
University of Illinois Chicago
Chicago, USA
psyu@uic.edu

Abstract

Recommending cold items remains a significant challenge in billion-scale online recommendation systems. While warm items benefit from historical user behaviors, cold items rely solely on content features, limiting their recommendation performance and impacting user experience and revenue. Current models generate synthetic behavioral embeddings from content features but fail to address the core issue: the absence of historical behavior data. To tackle this, we introduce the **LLM Simulator** framework, which leverages large language models to simulate user interactions for cold items, fundamentally addressing the cold-start problem. However, simply using LLM to traverse all users can introduce significant complexity in billion-scale systems. To manage the computational complexity, we propose a **coupled funnel ColdLLM** framework for online recommendation. ColdLLM efficiently reduces the number of candidate users from billions to hundreds using a trained coupled filter, allowing the LLM to operate efficiently and effectively on the filtered set. Extensive experiments show that ColdLLM significantly surpasses baselines in cold-start recommendations, including Recall and NDCG metrics. A two-week A/B test also validates that ColdLLM can effectively increase the cold-start period GMV.

CCS Concepts

• **Human-centered computing** → **Collaborative and social computing**; • **Information systems** → **Recommender systems**.

Keywords

cold-start recommendation, large language models, data mining

*Corresponding author.

This work is licensed under a Creative Commons Attribution International 4.0 License.

WSDM '25, March 10–14, 2025, Hannover, Germany
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1329-3/25/03
https://doi.org/10.1145/3701551.3703546

ACM Reference Format:
Feiran Huang, Yuanchen Bei, Zhenghang Yang, Junyi Jiang, Hao Chen, Qijie Shen, Senzhang Wang, Fakhri Karray, and Philip S. Yu. 2025. Large Language Model Simulator for Cold-Start Recommendation. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25), March 10–14, 2025, Hannover, Germany.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3701551.3703546

1 Introduction

Recommending cold items is essential for modern recommender systems, as a continuous flow of new content is being generated by individuals, companies, and AI [14, 31, 37]. Current large-scale recommender systems rely on historical user-item behaviors to learn user and item embeddings and then use these embeddings for downstream recall [11, 12, 30, 38] and CTR prediction tasks [5, 35, 40, 41]. However, unlike items with user historical interactions, or "warm" items, newly added items, or "cold" items lack behavior data to train embeddings. This lack of behaviors hinders the effective recommendation of cold items to users, impacting the overall ecosystem and the revenue of the recommender system. It is crucial to provide cold items with an embedding to ensure that these items have promising recommendation performance.

Current models typically use the content feature of cold items to generate synthetic embeddings [42–44]. In particular, **generative models** attempt to train a mapping function to ensure that the generated embedding approximates the behavior embedding. Representatively, DeepMusic [25] accomplishes this by minimizing the discrepancy between the generated embeddings and the actual behavioral embeddings. Expanding on this concept, GAR [6] employs a generative adversarial approach to ensure that the generated embeddings match the distribution of actual behavioral embeddings. ALDI [14] adopts actual behavioral embeddings as teachers to transfer their knowledge to the generated embeddings. Another line of models, **dropout models** further enhance the adaptability of recommendation models by incorporating both the generated embeddings and the behavior embeddings. For example, DropoutNet [24]

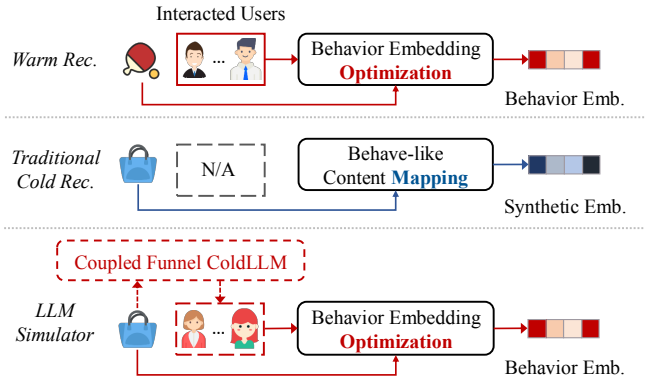

Figure 1: A comparison between traditional item cold-start models and our ColdLLM.

and Heater [45] typically drop random behavior embeddings and use the generated embeddings instead of real embeddings during training to improve robustness. CLCRec [33] utilizes contrastive learning to enhance the compatibility of the generated embeddings and the behavior embeddings.

However, existing solutions do not fully address the fundamental problem of cold-start—the lack of behavior data for cold items—which makes cold items inherently different from warm items. As illustrated in Figure 1, this fundamental difference leads to the following three key limitations:

(1) **Content-behavior gap:** The synthetic embeddings are still derived from content features. This approach results in a significant discrepancy between these synthetic embeddings and the embeddings learned from actual user behavior and interactions.
(2) **Suboptimal cold recommendation performance:** Current cold-start models often focus on recommending cold items alongside warm items, without significantly impacting warm items [14], less considering improving the recommendation performance for warm items.
(3) **Conflation of content-based and behavior-based recommendations:** Existing cold-start models typically conduct a mixed recommendation that mixes both content feature embeddings and behavioral embeddings.

Large Language Models (LLMs) show potential in addressing the aforementioned limitations, as they may be capable of understanding user preferences from content features and predicting users' intentions toward items [17, 34]. However, applying LLMs to cold-start item recommendations presents the following challenges:

(1) **Simulation of Cold-Start Behavior:** Training an LLM to predict a user's intention towards an item without actual interaction data is a challenge.
(2) **Efficiency of Simulation:** LLMs face constraints in inference efficiency. Simulating user behaviors for cold items across a large user base incurs substantial computational complexity.
(3) **Scalability to Large-Scale Recommendations:** There is an absence of mature frameworks leveraging LLMs to address the cold-start item issue in large-scale recommender systems.

Therefore, in this paper, we propose a novel **LLM Simulator** framework to fundamentally address the item cold-start problem. To tackle design challenges, we introduce the tailored structure of the LLM simulator, which includes user context construction, prompt design, and the simulation process. To accelerate the simulation process, we propose the **ColdLLM** for online recommendation, which efficiently scales down candidate users from billions to hundreds using a trained coupled filter. This filter is trained in conjunction with LLM to support its simulation. We then detail the fine-tuning process of the LLM and the coupled training of the filter model. Lastly, we present the implementation details of ColdLLM on large-scale recommender systems and analyze its complexity. The key contributions of this study can be summarized as follows:

- We formally define the behavior simulation problem and present a novel LLM Simulator framework that fundamentally addresses the cold-start recommendation issue.
- We propose a tailored training strategy for the simulator and offer a customized application strategy for the LLM simulator to the online recommender.
- We conduct extensive offline experiments, demonstrating that our model outperforms existing solutions by 21.69% in cold recommendation performance. A two-week A/B test further validates ColdLLM's effectiveness.

2 Related Works

2.1 Cold-Start Item Recommendation

Cold-start item recommendations refer to recommending newly occurred items to users. It presents a long-term challenge for recommendation systems due to the lack of behavioral interactions to model these items [1, 14, 18, 33].

Currently, one popular type of item cold-start recommendation model typically maps the contents of those cold items to content embeddings and then aligns them with the behavioral embeddings trained on warm items, which can be summarized as the "*embedding simulation*". Among them, one category of methods is the robust co-training models, which aims to align the behavioral embedding of warm items with the content-generated embedding of cold items through co-training with robust strategies [8, 23, 26, 33, 36, 45]. Another category is the knowledge alignment model, where the goal is to align the embeddings generated from the content of cold items towards the pre-trained behavioral embeddings based on those warm instances [6, 14, 20, 25]. Then, few other efforts have paid attention to the "*interaction simulation*", which generates some potential meaningful interactions between cold items and warm users/items [3, 18, 19, 29]. Representatively, UCC [19] generates low-uncertainty interactions for cold items that have a similar distribution to warm items with teacher-student consistency learning. MI-GCN [29] adopts pair-wise mutual information to generate informative interactions for cold items before the GNN convolution. USIM [18] further proposes a user behavior imagination framework with a reinforcement learning strategy for cold items.

2.2 Recommendation with LLMs

Recently, Large Language Models (LLMs) have emerged as a central research focus due to their remarkable ability to understand and

generate human-like text, leveraging their pre-trained repository of world knowledge [4, 9, 39].

Given the rich tapestry of natural language descriptions inherent in recommender systems, an increasing number of studies are honing in on the potential of LLMs to enhance recommendation capabilities [2, 16, 22, 32]. Representatively, TALLRec [2] introduces a novel framework that adeptly integrates LLMs with recommendation tasks through a dual-stage tuning process. TALLRec has been shown to improve recommendation performance and robust cross-domain recommendation tasks. LLMRec [32] enhances the recommendation performance by strengthening user-item interaction links, enriching item features, and profiling users from a natural language vantage point. Additionally, the work presented in [28] pioneers the use of LLMs to address the cold-start challenge in recommendations, employing the models as data augmenters to generate training signals for cold items through prompting.

In this paper, we focus on leveraging the world knowledge of LLMs and the collaborative filtering capabilities of recommendation models for cold-start item recommendations.

3 Preliminaries

Notations. The user and item sets are denoted as $\mathcal{U}$ and $\mathcal{I}$, respectively. In terms of the items, we denote the warm items (items with historical interactions) as $\mathcal{I}_w$, and the cold items (items without historical interactions) as $\mathcal{I}_c$. Let $\mathcal{H}$ present the set of all the historically interacted user sequences for all the items. Then, each warm item has an interacted user sequence $\boldsymbol{s}_i = \{h_{i,1}, \cdots, h_{i,|h_i|}\}$, $\forall i \in \mathcal{I}_w$, where $|h_i|$ denotes the number of interaction for item i. For a cold item j, the interacted user sequence is a null set, namely $\boldsymbol{s}_j = \phi$. With the historically interacted user-item pairs, we can learn the behavioral embedding vectors for each user and warm item, namely, $\boldsymbol{e}_u$, $\forall u \in \mathcal{U}$ and $\boldsymbol{e}_i$, $\forall i \in \mathcal{I}_w$. We have C to denote the contents of the items, and each item has its respective content features, denoted as $\boldsymbol{c}_i$. For the user, we gather the item content list, denoted as $\boldsymbol{C}_u$.

Restrict Item Cold-start Recommendation. This paper focuses on the most challenging strict cold-start problem, from the view of item cold-start, where the cold items lack any historical behaviors. Under this constraint, the warm items and the cold items lead to two different ways of recommendation. The warm items are recommended with historical user sequences, which are usually encoded into behavior embeddings. Formally, the warm recommendation can be defined as:

$$\hat{y}_{ui}^{(w)} = R(\boldsymbol{e}_u, \mathrm{Emb}_{cf}(\boldsymbol{s}_i), \boldsymbol{c}_i), \ i \in \mathcal{I}_w, \tag{1}$$

where $\mathrm{Emb}_{cf}(\cdot)$ denote the collaborative filtering function for behavior embedding. However, the user sequence set of the cold item is empty, making the cold items recommendation to be organized with the following formula:

$$\hat{y}_{ui}^{(c)} = R(\boldsymbol{e}_u, \mathrm{Emb}_{cf}(\phi), \boldsymbol{c}_i), \ i \in \mathcal{I}_c. \tag{2}$$

Thus the restricted cold-start recommendation problem turns to recommend the above warm items and cold items well.

4 Methodology

In this section, we first introduce the general framework of our proposed ColdLLM. Next, we assume the existence of a trained LLM simulator and explain how to simulate user sequences using the coupled funnel strategy, which includes coupled filtering simulation and coupled refining simulation. Following this, we elaborate on the training strategy of the LLM and the filter simulator. Lastly, we outline the industrial implementation details and provide a complexity analysis of ColdLLM.

4.1 Overall Framework

The primary distinction between cold and warm items lies in their interaction histories: cold items lack historical behaviors, while warm items have rich interaction data. Traditional models mainly address this cold-start problem through synthetic embedding construction approaches, which can introduce a natural gap between cold and warm items [6, 14]. One fundamental solution is to simulate user behaviors for each cold item, and then obtain the cold item embedding from behavior embedding optimization rather than from a mapping function. Building on this concept, we introduce behavior simulation and embedding optimization.

4.1.1 Behavior Simulation. The behavior simulation summarizes all the historical behaviors and all the user and item information to simulate possible users that can aid in updating the embeddings for cold items. Considering this, we employ LLM to analyze all the positive historical behaviors to act as a simulator for generating user sequences for cold items. Specifically, the ColdLLM process can be formally defined as follows:

$$\hat{\mathbf{s}}_i = \mathrm{ColdLLM}(\boldsymbol{c}_i, \mathcal{U}, \mathcal{H}, C), \forall \ i \in \mathcal{I}_c. \tag{3}$$

In the ideal scenario, the ColdLLM could have access to the information of cold item $\boldsymbol{c}_i$, the entire user set $\mathcal{U}$, the complete historical interactions of all users $\mathcal{H}$, and the content details of all items C.

4.1.2 Embedding Optimization. By simulating the user sequence for cold items, these items are transformed into warm items. Through simulated behaviors, the recommender system utilizes the existing behavior embedding optimization structure to leverage trained user and warm item embeddings for optimizing the cold item embedding. In offline datasets, such simulation can even enrich the training data to enhance user representation further. For online large-scale platforms, simulated interactions are used solely to update the cold item embedding. The **final cold embedding** for downstream tasks can be formally presented as follows:

$$\boldsymbol{e}_i^{(c)} = \mathrm{Emb}_{\mathrm{opt}}(\boldsymbol{c}_i, \hat{\mathbf{s}}_i, \boldsymbol{E}), \tag{4}$$

where $\mathrm{Emb}_{\mathrm{opt}}(\cdot)$ denotes the general behavior embedding optimizer of the recommendation systems, $\boldsymbol{e}_i^{(c)}$ represents the embedding of the cold item i, and $\boldsymbol{s}_i^{(c)}$ is the simulated user sequence for the cold item. $\boldsymbol{E}$ denotes all the trained warm embeddings, including the users and the warm items.

4.2 Coupled Funnel ColdLLM

Though Eq. (3) provides an ideal case of an LLM simulator, compared with traditional embedding-based models, the LLM suffers from heavy computational complexity, requiring more expensive GPUs.

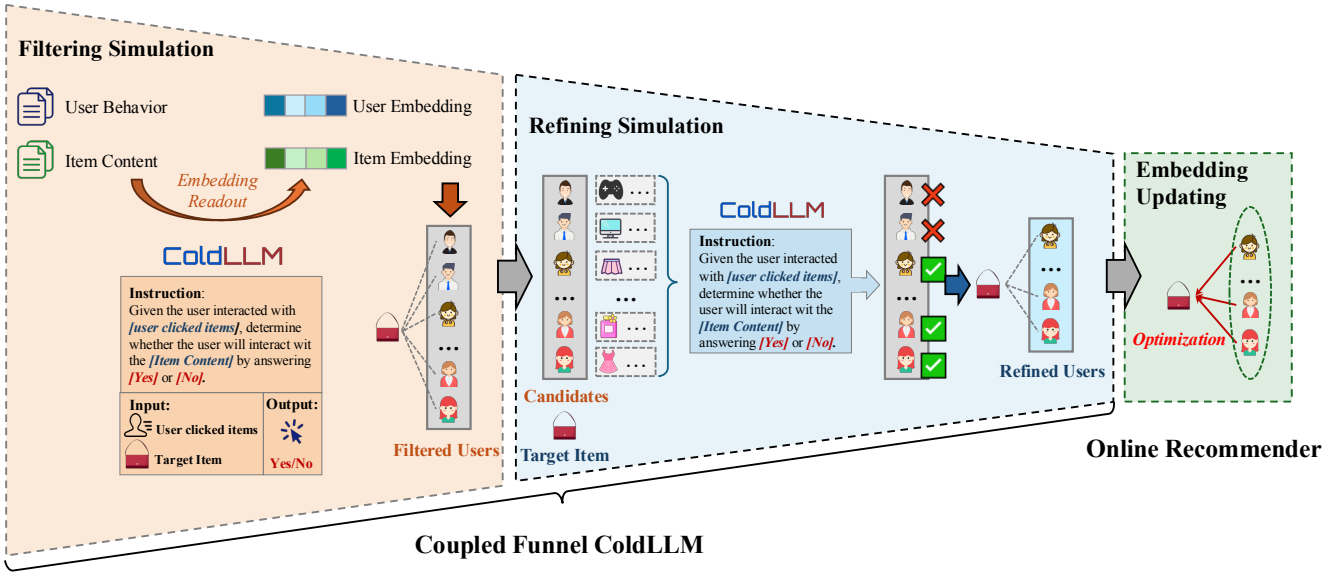

Figure 2: The overall model architecture of the ColdLLM.

This makes the original ColdLLM unsuitable for billion-scale recommendations. In this subsection, we propose the coupled-funnel ColdLLM to incorporate coupled filter models efficiently and effectively simulate cold item behaviors.

4.2.1 ***Filtering Simulation.*** The aim of the filtering process is to diminish potential users from a dataset of billion-scale to a more manageable range of tens to thousands. Embedding-based filtering models and LLMs encounter distinct challenges in forecasting user behaviors toward cold items. Embedding-based filtering models adaptly embed users and items into vectors but encounter difficulty in capturing users' content-level intent and items' high-level content information. To address this, we initially enhance the filtering models with LLM-processed embeddings, and in the subsequent subsection, we present the coupled training of the filtering model.

Formally, we employ an LLM to extract the content embedding of an item and then apply a matching function to map this embedding for behavior filtering, which can be expressed as follows:

$$\boldsymbol{f}_i = \mathcal{F}_{\mathcal{I}}(LLM_{emb}(\boldsymbol{c}_i)), \tag{5}$$

where $\boldsymbol{f}_i$ represents the filtering embedding for item i, $\mathcal{F}_{\mathcal{I}}(\cdot)$ is the mapping function, and $LLM_{emb}(\cdot)$ is the LLM embedding readout function.

The embedding readout function is designed to extract the content embedding from the LLM. Specifically, we first obtain the last layer embedding, which represents the processed token information, and then apply mean pooling to derive the content feature embedding for any given cold item:

$$\boldsymbol{f}_i = \mathcal{F}_{\mathcal{I}}\left(\frac{1}{|\boldsymbol{c}_i|}\sum_{j=1}^{|\boldsymbol{c}_i|}\mathcal{E}^{(L)}(\boldsymbol{c}_i)[j]\right), \tag{6}$$

where $\mathcal{E}^{(L)}(\boldsymbol{c}_i)[j]$ represents the j-th embedding of the L-th layer of the LLM. Here, $\boldsymbol{c}_i$ stands for the content feature of the item, $\boldsymbol{c}_i[j]$ refers to the j-th token in $\boldsymbol{c}_i$, and $|\boldsymbol{c}_i|$ indicates the total number of tokens in $\boldsymbol{c}_i$.

To filter users who are likely to interact with the cold item, we consider both content embeddings and behavioral embeddings. We use the dot product of the mapped user embedding and the mapped item embedding to identify the top-K highest score candidates:

$$s_i^{(f)} = \mathrm{Top}_K\left(\{\mathcal{F}_{\mathcal{U}}(\boldsymbol{e}_u|C_u)^\top \cdot \boldsymbol{f}_i \mid \forall u \in \mathcal{U}\}\right), \tag{7}$$

where $\mathcal{F}_{\mathcal{U}}$ is the mapping function for the users. Please refer to the following subsection for design and training details.

This top-K computation can be accelerated using efficient similarity search platforms, such as FAISS [15], resulting in $O(1)$ computational complexity.

4.2.2 Refining Simulation. After filtering, the candidate user pool will decrease from billions to tens or hundreds. Next, we employ LLMs for examination and enhancement. For each iteration, we feed the user's context and the item's content into the LLM, which outputs a prediction of whether the user will interact with the item, displaying 'yes' or 'no'. The refining module considers three technique details aspects: 1. Context construction, 2. Prompt Design, and 3. Refining Process.

Context Construction. LLMs rely on users's context to judge whether the user will interact with the recommendation. However, the users may have too many historical behaviors, or not all the historical behaviors are correlated to the query item. To this end, we utilize the item embedding from the filtering process to filter the related items,

$$C_u^{(f)} = \mathrm{Top}_L\left(\{\boldsymbol{f}_i^\top \cdot \boldsymbol{f}_j \mid \forall j \in C_u\}\right), \tag{8}$$

where C_u denotes the user's historically interacted items.

Prompt Design. The LLM prompt contains three parts: 1. fixed prompts, 2. user context, and 3. item content. The fixed prompts are

used to set up the goal of the LLM Simulator. As shown in Figure 2, the fixed prompt is

> "Given the user interacted with [Text($C_u^{(f)}$)], determine whether the user will interacted the [Text(c_i)] by answering Yes or No."

Then given an LLM, the pair-wise LLM simulation can be present as follows:

$$\hat{Z}_{u,i} = \begin{cases} 1, & \text{LLM}(C_u^{(f)}, c_i) = \text{"}Yes\text{"} \\ 0, & \text{LLM}(C_u^{(f)}, c_i) = \text{"}No\text{"} \end{cases}, \tag{9}$$

where $\hat{Z}_{u,i}$ represents the predicted value of the LLM. Subsequent research could consider employing a more intricate framework to obtain a continuous value.

Refine Process. During the refining process, we traverse the filtered user set and only maintain the users that are predicted as "yes" by the LLM simulator. Thus finally, the simulated users can be obtained as follows,

$$s_i^{(r)} = \{u | \hat{Z}_{u,i} = 1 \mid \forall u \in s_i^{(f)}\}. \tag{10}$$

After obtaining the refined user simulation results, the sequence can be fed into the behavior embedding optimization framework (Eq. (4)) to enhance the cold item embedding.

4.3 Simulator Training

In this subsection, we introduce the simulator training, including the training of the based LLM model and the filtering model.

4.3.1 LLM Training. To better fit the recommendation data of each different recommendation scenario, we utilize a Low-Rank fine-tuning strategy to ensure the LLM can capture the data distribution of the recommendation scenario, and capture the trends by performing online model updating.

Data Preparation. We use user item behaviors to train our LLM. In the online recommendation, we encounter three types of behaviors: positive behaviors (users click on an item), negative behaviors (users ignore the item), and unobserved behavior (the recommender system does not recommend the item to the user, and we cannot observe the user's intention).

As negative behaviors make up approximately 70%-90% of online behaviors, we perform a 1:1 sampling of positive and negative behaviors to address the imbalanced distribution. Additionally, we conduct an extra 1:1 sampling of positive and unobserved behavior to ensure that the LLM can capture a broader range of intentions, especially for cold items. For offline datasets, we only use a 1:1 sampling of positive and unobserved data to train the LLM.

Fine-tuning Structure. Specifically, we add an additional low-rank modification matrix on all the parameters of the transformer structure, in terms of Q, K, and V in the self-attention mechanism, as well as the feed-forward layers. This Low-Rank Adaptation (LoRA) [13] approach allows us to efficiently fine-tune the large language model for specific recommendation scenarios while maintaining most of the pre-trained weights. Specifically, we add trainable rank decomposition matrices to each weight matrix of the original model. For a given weight matrix $W \in \mathbb{R}^{d\times g}$, we add a low-rank update:

$$W' = W + \Delta W = W + BA, \tag{11}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times g}$ are the low-rank decomposition matrices, with r representing the rank of the decomposition, which is generally much smaller than the dimensions d and g. The update on the transformer networks can be written as follows:

$$Q = W_Q x + \Delta W_Q x = W_Q x + B_Q A_Q x, \tag{12}$$

$$K = W_K x + \Delta W_K x = W_K x + B_K A_K x, \tag{13}$$

$$V = W_V x + \Delta W_V x = W_V x + B_V A_V x, \tag{14}$$

where W_Q, W_K, and W_V are the original weight matrices for Query, Key, and Value projections, B_Q, B_K, B_V and A_Q, A_K, A_V are the low-rank decomposition matrices for each projection.

Similarly, for the feed-forward layers, we have:

$$\text{FFN}_{\text{LoRA}}(x) = \text{ReLU}(x(W_1 + A_1B_1) + b_1)(W_2 + A_2B_2) + b_2, \tag{15}$$

where B_1, A_1, B_2, and A_2 are the low-rank decomposition matrices for the FFN.

4.3.2 Coupled Filter Model Training. The coupled filter model has two design proposals: 1. reflect user item behaviors; 2. coupled with the LLM. Specifically, we utilize the combination of two pairs of embedding to accomplish this purpose.

Training of the Behavior Filtering. For every given user-item pair (u, i), a negative pair (u, j) is randomly selected. These pairs can be collectively represented as a triple (u, i, j). The output of behavior filtering can be expressed as $\hat{Y}_{ui}^{(B)} = \mathcal{F}_{\mathcal{U}}^{(B)}(\boldsymbol{e}_u)^\top \cdot \mathcal{F}_{\mathcal{I}}^{(B)}(LLM_{emb}(\boldsymbol{c}_i))$. We consider BPR loss [21] to optimize the recommendation performance of the behavior filtering model

$$\mathcal{L}_{BPR} = -\sum_{(u,i,j)} \ln \sigma(\hat{Y}_{ui}^{(B)} - \hat{Y}_{uj}^{(B)}), \tag{16}$$

where $\sigma(\cdot)$ is the sigmoid function. This loss encourages the filter model to rank the positive item higher than the negative item. Besides, we also utilize the aligning loss in ALDI [14] to help the training of behavior filtering.

Training of the Coupled ColdLLM Filtering. For the coupled LLM filtering, we apply the formula below to filter the users: $\hat{Y}_{ui}^{(L)} = \boldsymbol{F}_{\boldsymbol{U}}^{(L)}(\boldsymbol{e}_u)^\top \cdot \boldsymbol{F}_{\boldsymbol{I}}^{(L)}(LLM_{emb}(\boldsymbol{c}_i))$ In addition to the BPR loss, we introduce the coupled ColdLLM loss to maintain similarity with the ColdLLM in the coupled filter model:

$$\mathcal{L}_{coupled} = -\sum_{(u,i)} \left(\hat{Z}_{ui} \ln \hat{Y}_{ui}^{(L)} + (1 - \hat{Z}_{ui}) \ln(1 - \hat{Y}_{ui}^{(L)})\right). \tag{17}$$

4.4 Implementation Strategy

ColdLLM has demonstrated its effectiveness by providing helpful cold-start recommendations on the e-commerce platform. In this subsection, we detail the online deployment of ColdLLM and analyze its computational complexity.

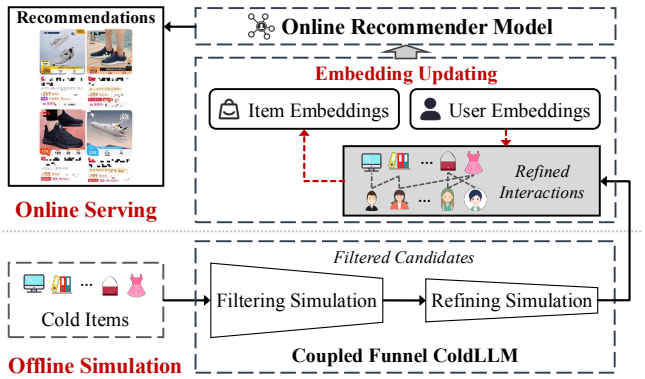

Figure 3: The system architecture for ColdLLM deployment.

Real-World Deployment Strategy. As shown in Figure 3, our overall framework for ColdLLM consists of three primary components: (i) online serving; (ii) online training (embedding updating); and (iii) offline simulation.

When new items are uploaded to the platform, we first employ our model to simulate user interactions for embedding updates. These simulated user-item pairs are then fed into the online embedding updating structure. Since these interactions are simulated rather than actual user behaviors, we update only the embeddings of the cold items. Finally, we transmit the updated cold item embeddings to the online recommendation service.

Complexity Analysis. The computational complexity of ColdLLM comprises three main components: coupled filtering complexity, coupled refining complexity, and embedding update complexity.

(1) **Coupled Filtering:** Leveraging similarity indexing frameworks like FAISS, we can efficiently downscale the candidate users from billions to hundreds with a complexity of $O(1)$ in approximately 60 ms.
(2) **Coupled Refining:** We refine the filtered candidates using fine-tuned LLaMA-7B models to identify 20 qualified users. This process takes about 200-400 ms for each user-item pair. In total, the LLM-refining stage requires less than 8 seconds.
(3) **Embedding Update:** The online embedding process utilizes the simulated interactions to optimize the cold item embeddings within 120 ms.

To enhance efficiency, we employ parallel processing techniques. When equipped with three 8×A100 GPU machines, our system can handle a load of 8,640 cold items per hour. This capacity can be easily scaled by adding more GPUs to the infrastructure.

5 Experiments

In this section, we conduct comprehensive experiments on benchmark cold-start recommendation datasets, aiming to answer the following research questions. **RQ1:** Does ColdLLM outperform contemporary state-of-the-art cold-start recommendation models in overall, warm, and cold recommendations? **RQ2:** What is the effect of different components in ColdLLM? **RQ3:** How do key hyperparameters impact the performance of ColdLLM? **RQ4:** How does ColdLLM perform in the real-world industrial recommendations?

5.1 Experimental Setup

5.1.1 Datasets. We conduct experiments on two widely used datasets: **CiteULike**[1] [27], containing 5,551 users, 16,980 articles, and 204,986 interactions, and **MovieLens**[2] [10], comprising 6,040 users, 3,883 items, and 1,000,210 interactions. For each dataset, following previous works [14], 20% items are designated as cold-start items, with interactions split into a cold validation set and testing set (1:1 ratio). Records of the remaining 80% of items are divided into training, validation, and testing sets, using an 8:1:1 ratio.

5.1.2 Compared Baselines. To assess the effectiveness of our proposed ColdLLM, we conducted a comprehensive analysis with ten leading-edge models in the domain of cold-start item recommendations, which can be categorized into three main groups. (i) Dropout-based embedding simulation models: **DropoutNet** [26], **MTPR** [8], and **CLCRec** [33]. (ii) Generative-based embedding simulation models: **DeepMusic** [25], **MetaEmb** [20], **GNP** [7], **GAR** [6], and **ALDI** [14]. (iii) Behavior simulation models: **UCC** [19] and **MI-GCN** [29]. To further verify the universality of ColdLLM, we verify these models on three widely used recommendation backbones: **MF** [21], **NGCF** [30], and **LightGCN** [11].

5.1.3 Hyperparameter Setting. In the filtering phase, we utilized AdamW as the optimizer with a chosen learning rate of 1×10^{-5}, and set the batch size for each training batch to 128. We opted for a top-k value of 20. In the refining phase, the learning rate was adjusted to 5×10^{-5}. The dimension of the embeddings was standardized to 200 for all models. We employed the Adam optimizer with a learning rate of 1×10^{-3} and applied early stopping by monitoring NDCG on the validation set.

5.1.4 Evaluation Metrics. Our evaluation encompasses the overall, warm, and cold recommendation performance, adopting a widely adopted full-ranking evaluation approach [11, 14]. Specifically, we employ Recall@K and NDCG@K as our primary metrics, where $k = 20$. Following previous works [6, 14], during the testing, we randomly select 2,000 users for evaluation.

5.2 Main Results (RQ1)

The performance comparison of overall, warm, and cold recommendations between ColdLLM and other baselines on benchmark datasets is presented in Table 1. From the results, we can have the following observations:

ColdLLM can achieve significant improvements over current methods. From the table, we can find that ColdLLM can consistently demonstrate superiority across different datasets and backbones. Specifically, ColdLLM brings an average NDCG improvement of 10.79% and 37.10% on overall and cold item recommendations over LightGCN. This enhancement illustrates the effectiveness of ColdLLM with the coupled-funnel behavior simulation based on the LLM's world knowledge.

The generative-based embedding simulation models generally perform better in warm and overall recommendations than dropout-based embedding simulation models. This indicates that forcing the warm behavior embeddings and cold content

[1]https://github.com/js05212/citeulike-a
[2]https://grouplens.org/datasets/movielens/10m

Table 1: Comparison results on overall, cold, and warm item recommendations over three backbone models (MF, NGCF, LightGCN). The best and second-best results in each column are highlighted in bold font and underlined.

	Method	Overall Recommendation				Cold Recommendation				Warm Recommendation			
		CiteULike		MovieLens		CiteULike		MovieLens		CiteULike		MovieLens	
		Recall	NDCG	Recall	NDCG	Recall	NDCG	Recall	NDCG	Recall	NDCG	Recall	NDCG
MF	DropoutNet	0.0794	0.0670	0.0646	0.1127	0.2268	0.1356	0.0671	0.0808	0.1343	0.0792	0.1391	0.1386
	MTPR	0.1060	0.0810	0.0739	0.1055	0.2496	0.1476	0.0745	0.0811	0.1728	0.0998	0.1628	0.1388
	CLCRec	0.1269	0.0992	0.0784	0.1358	0.2295	0.1347	0.0744	0.0726	0.1898	0.1167	0.1699	0.1696
	DeepMusic	0.0956	0.0789	0.0933	0.1377	0.2141	0.1262	0.0327	0.0391	0.2838	0.1933	0.2076	0.1819
	MetaEmb	0.0972	0.0804	0.0933	0.1377	0.2232	0.1306	0.0432	0.0468	0.2838	0.1933	0.2076	0.1819
	GNP	0.1568	0.1305	0.0813	0.1253	0.2107	0.1193	0.0683	0.0704	0.2838	0.1993	0.2076	0.1819
	GAR	0.1440	0.1132	0.0462	0.0812	0.2453	0.1479	0.0348	0.0510	0.2272	0.1438	0.1003	0.1003
	ALDI	0.1618	0.1204	0.0914	0.1355	0.2684	0.1550	0.0431	0.0464	0.2838	0.1993	0.2076	0.1819
	ColdLLM	**0.2213**	**0.1599**	**0.0976**	**0.1473**	**0.3218**	**0.1875**	**0.1225**	**0.1182**	**0.3058**	**0.2061**	**0.2129**	**0.1924**
	Improvement	36.77%	22.52%	4.61%	6.98%	19.90%	20.96%	64.43%	45.75%	7.75%	3.41%	2.56%	5.78%
NGCF	DropoutNet	0.0813	0.0656	0.1144	0.1935	0.2211	0.1278	0.0214	0.0223	0.1416	0.0842	0.2517	0.2457
	MTPR	0.1006	0.0769	0.1132	0.1818	0.2479	0.1391	0.0749	0.0894	0.1753	0.0977	0.2509	0.2329
	CLCRec	0.1201	0.0920	0.1270	0.2042	0.2093	0.1188	0.0694	0.0777	0.1886	0.1136	0.2807	0.2627
	DeepMusic	0.1269	0.1043	0.1393	0.2265	0.1980	0.1152	0.0409	0.0493	0.2347	0.1485	0.3076	0.2922
	MetaEmb	0.1119	0.0957	0.1393	0.2265	0.2830	0.1664	0.0247	0.0245	0.2347	0.1485	0.3076	0.2922
	GNP	0.1449	0.1099	0.1381	0.2234	0.2318	0.1371	0.0589	0.0646	0.2347	0.1485	0.3076	0.2922
	GAR	0.1144	0.0909	0.0098	0.0174	0.2099	0.1251	0.0166	0.0178	0.1850	0.1143	0.2788	0.2721
	ALDI	0.1541	0.1141	0.1393	0.2206	0.2466	0.1399	0.1022	0.1113	0.2347	0.1485	0.3076	0.2922
	UCC	0.1094	0.0990	0.0981	0.1523	0.0019	0.0008	0.0063	0.0066	0.2402	0.1582	0.2191	0.1999
	MI-GCN	0.1324	0.1158	0.1401	0.2299	0.0641	0.0335	0.0162	0.0187	0.2827	0.1815	0.3104	0.2973
	ColdLLM	**0.2211**	**0.1653**	**0.1447**	**0.2372**	**0.3404**	**0.1986**	**0.1670**	**0.1662**	**0.2992**	**0.1958**	**0.3186**	**0.3052**
	Improvement	43.48%	42.75%	3.28%	3.18%	20.28%	19.35%	63.41%	49.33%	5.84%	7.88%	2.64%	2.66%
LightGCN	DropoutNet	0.0883	0.0639	0.1165	0.1978	0.2309	0.1312	0.0340	0.0373	0.1175	0.0692	0.2560	0.2518
	MTPR	0.1001	0.0753	0.1011	.0.1551	0.2585	0.1454	0.0779	0.0802	0.1753	0.0697	0.2247	0.2009
	CLCRec	0.1293	0.0965	0.1253	0.2037	0.2435	0.1425	0.0677	0.0816	0.2149	0.1302	0.2764	0.2612
	DeepMusic	0.0985	0.0745	0.1418	0.2330	0.2239	0.1259	0.0635	0.0719	0.2528	0.1541	0.3130	0.3008
	MetaEmb	0.0924	0.0714	0.1418	0.2330	0.2252	0.1295	0.0248	0.0244	0.2528	0.1541	0.3130	0.3008
	GNP	0.1609	0.1197	0.1293	0.2106	0.2606	0.1532	0.0771	0.0760	0.2528	0.1541	0.3130	0.3008
	GAR	0.1357	0.1062	0.0106	0.0195	0.2539	0.1489	0.0110	0.0130	0.2339	0.1455	0.2873	0.2794
	ALDI	0.1626	0.1201	0.1428	0.2316	0.2692	0.1539	0.1229	0.1295	0.2528	0.1541	0.3130	0.3008
	UCC	0.1374	0.1260	0.1277	0.2020	0.0020	0.0011	0.0063	0.0073	0.3002	0.2010	0.2830	0.2641
	MI-GCN	0.1557	0.1483	0.1454	0.2378	0.0507	0.0262	0.0307	0.0312	**0.3372**	**0.2362**	0.3227	0.3068
	ColdLLM	**0.2285**	**0.1747**	**0.1506**	**0.2468**	**0.3601**	**0.2126**	**0.1759**	**0.1762**	0.3252	0.2156	**0.3314**	**0.3186**
	Improvement	40.52%	17.80%	3.58%	3.78%	33.76%	38.14%	43.12%	36.06%	–	–	2.70%	3.85%

embeddings to align with each other through the same embedding layer may lead to a performance drop in the warm item recommendation. The interaction simulation with ColdLLM addresses this by allowing cold and warm items to be adequately trained within a unified recommender.

Existing behavior simulation models retain relatively good performance in overall and warm recommendations, but fall short in cold recommendations. A possible reason is that the behavior generation only based on the content information with DNNs is insufficient for accurate behavior simulation for cold items. ColdLLM leverages the world knowledge of LLMs to utilize more information for the behavior simulation of cold items.

5.3 Ablation Study (RQ2)

We conduct an ablation study of our proposed ColdLLM approach to validate its key components, of which the results are illustrated in Figure 4. Specifically, we compare ColdLLM with its five variants: (i) ***w/o LSF & R*** removes the coupled ColdLLM filtering and the refining simulation. (ii) ***w/o BF & R*** removes the behavior filtering and the refining simulation. (iii) ***w/o LSF*** skips the coupled ColdLLM filtering module. (iv) ***w/o BF*** skips the behavior filtering module. (v) ***w/o R*** skips the refining simulation. Further, we compare the adoption rate of the filtered users by the filtering simulation of ColdLLM with three different strategies, as shown in Table 2. From the results, we can have the following observations:

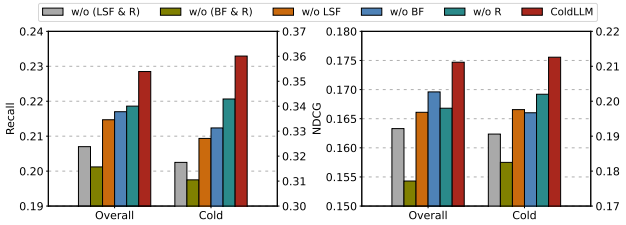

Figure 4: Ablation study results on CiteULike.

Table 2: Adoption rate of different filtering strategies.

Strategy	Random	w/o LSF	w/o BF	**ColdLLM**
Adoption rate (%)	8.18%	80.45%	81.33%	**85.96%**

Effectiveness of the filtering simulation. The decline in performance for *w/o LSF* and *w/o BF* demonstrates the effectiveness of the filtering simulation. Further, the more pronounced drop in performance for *w/o LSF* highlights the importance of world knowledge in LLMs for generating behavior patterns. Additionally, the adoption rates in Table 2 indicate that the filtering simulation yields behaviors of superior quality compared to alternative strategies.

Effectiveness of refining stage. The necessity of the refining stage is evident when comparing the performance with *w/o R*. Furthermore, the model *w/o LSF & R* and *w/o BF & R*, exhibit a more significant performance decline than *w/o LSF* and *w/o BF*. This contrast also underscores the meaningful contribution of the refining simulation to the overall effectiveness of ColdLLM.

5.4 Parameter Study (RQ3)

In this subsection, we study the impact of key hyper-parameters on ColdLLM with CiteULike, including the filtering candidate number K in Eq.(7) and the learning rate of embedding updating. The results are shown in Figure 5.

Effect of filtering candidate number K. From the results, we can find that optimal results for overall and warm recommendations are achieved with a modest value of K, such as K=10 for CiteULike. Conversely, a larger K is beneficial for cold recommendations, with K=50 for CiteULike yielding the best outcomes. However, an excessively large K can degrade performance by introducing noise from irrelevant interactions.

Effect of the updating learning rate. From the figure, we can observe that the three types of recommendation tasks achieve the best results with a similar optimal learning rate, which indicates that the tuning of the learning rate would simultaneously be suitable for all three tasks.

5.5 Online Evaluation (RQ4)

To validate the effectiveness of ColdLLM in an industrial setting, we conducted an online A/B test on a large-scale e-commerce platform. The experiment ran for two consecutive weeks, involving 5% of users for each group. We compared ColdLLM against three representative baselines: **Random**, **MetaEmb** [20], and **ALDI** [14]. Table 3 presents the results of these online A/B tests.

Evaluation Metrics. We employed three tailored metrics to assess the performance of ColdLLM against existing baselines:

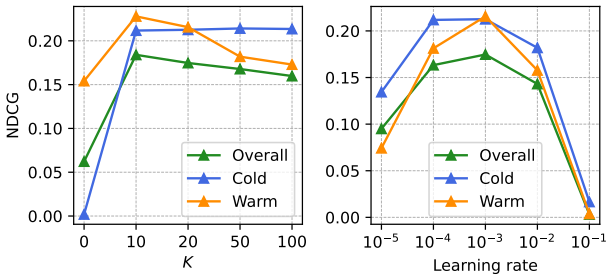

Figure 5: Parameter study results on CiteULike.

- Page Views (Cold-PV): The number of user clicks during the cold-start period.
- Page Click-Through Rate (Cold-PCTR): The ratio of clicks to impressions during the cold-start period.
- Gross Merchandise Value (Cold-GMV): The total value of user purchases during the cold-start period.

We define the cold-start period as the interval from the item's publication to two hours after its release.

Results and Analysis. The results in Table 3 demonstrate that ColdLLM consistently outperforms both baselines across all metrics. Specifically, compared to the random baseline, ColdLLM achieves substantial improvements of 11.45% in Cold-PV, 5.60% in Cold-PCTR, and 23.80% in Cold-GMV for cold items. When compared to MetaEmb, ColdLLM shows significant gains of 9.20% in Cold-PV, 4.35% in Cold-PCTR, and 18.25% in Cold-GMV. Even against the strong ALDI baseline, ColdLLM maintains superior performance with impressive improvements of 7.25% in Cold-PV, 3.70% in Cold-PCTR, and 16.90% in Cold-GMV.

These remarkable improvements across all metrics underscore the effectiveness of ColdLLM in addressing the item cold-start problem in real-world recommender systems. The consistent and substantial performance gains, particularly in Cold-GMV, highlight the practical impact of our approach on business outcomes in e-commerce settings.

Table 3: Results of online A/B tests.

A/B Test	Cold-PV	Cold-PCTR	Cold-GMV
vs. Random	+11.45%	+5.60%	+23.80%
vs. MetaEmb	+9.20%	+4.35%	+18.25%
vs. ALDI	+7.25%	+3.70%	+16.90%

6 Conclusion

In this paper, aiming to address current limitations, we propose ColdLLM, which fundamentally solves the cold-start problem in large-scale recommendation systems, significantly improving performance and economic impact. Both online and offline experiments verified the strength of our proposed ColdLLM. Based on the observation, ColdLLM opens new possibilities for leveraging large language models in large-scale online recommendations.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (No. 62272200, No. 62172443) and Hunan Provincial Natural Science Foundation of China (No. 2022JJ30053).

Ethical Considerations

Our paper focuses on leveraging LLMs to simulate the behaviors of cold items. We believe that pursuing this direction is essential, as it not only reveals the untapped potential of LLMs for a large amount of newly occurred items in recommender systems, while concurrently exhibiting the potential ethical considerations. Here we list two main considerations as follows:

(1) **Industrial Scenario**: We discuss the application in an industrial scenario, which will benefit industrial users and inspire communication between industrial researchers and academic researchers. Nevertheless, it may also pose computational and storage challenges for academic researchers.

(2) **LLM Resource Cost**: LLMs' training and deployment require significant computational power, potentially leading to higher energy consumption and environmental impact. Additionally, a minimum of one GTX 3090 may be required for the experiments.

References

[1] Haoyue Bai, Min Hou, Le Wu, Yonghui Yang, Kun Zhang, Richang Hong, and Meng Wang. 2023. Gorec: a generative cold-start recommendation framework. In *Proceedings of the 31st ACM international conference on multimedia*. 1004–1012.

[2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. *arXiv preprint arXiv:2305.00447* (2023).

[3] Desheng Cai, Shengsheng Qian, Quan Fang, Jun Hu, and Changsheng Xu. 2023. User cold-start recommendation via inductive heterogeneous graph neural network. *ACM Transactions on Information Systems (TOIS)* (2023).

[4] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.

[5] Hao Chen, Yuanchen Bei, Qijie Shen, Yue Xu, Sheng Zhou, Wenbing Huang, Feiran Huang, Senzhang Wang, and Xiao Huang. 2024. Macro graph neural networks for online billion-scale recommender systems. In *International World Wide Web Conference (WWW)*.

[6] Hao Chen, Zefan Wang, Feiran Huang, Xiao Huang, Yue Xu, Yishi Lin, Peng He, and Zhoujun Li. 2022. Generative adversarial framework for cold-start item recommendation. In *Conference on Information Retrieval (SIGIR)*.

[7] Hao Chen, Yu Yang, Yuanchen Bei, Zefan Wang, Yue Xu, and Feiran Huang. 2025. Graph Neural Patching for Cold-Start Recommendations. In *Australasian Database Conference*. Springer, 334–346.

[8] Xiaoyu Du, Xiang Wang, Xiangnan He, Zechao Li, Jinhui Tang, and Tat-Seng Chua. 2020. How to learn item representation for cold-start multimedia recommendation?. In *International Conference on Multimedia (MM)*.

[9] Mingqi Gao, Xinyu Hu, Jie Ruan, Xiao Pu, and Xiaojun Wan. 2024. Llm-based nlg evaluation: Current status and challenges. *arXiv preprint arXiv:2402.01383* (2024).

[10] F Maxwell Harper and Joseph A Konstan. 2015. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TIIS)* (2015).

[11] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Conference on Information Retrieval (SIGIR)*.

[12] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *International World Wide Web Conference (WWW)*.

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[14] Feiran Huang, Zefan Wang, Xiao Huang, Yufeng Qian, Zhetao Li, and Hao Chen. 2023. Aligning Distillation For Cold-Start Item Recommendation. In *Conference on Information Retrieval (SIGIR)*.

[15] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (2019).

[16] Lei Li, Yongfeng Zhang, Dugang Liu, and Li Chen. 2023. Large language models for generative recommendation: A survey and visionary discussions. *arXiv preprint arXiv:2309.01157* (2023).

[17] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. *arXiv preprint arXiv:2306.05817* (2023).

[18] Ruochen Liu, Hao Chen, Yuanchen Bei, Qijie Shen, Fangwei Zhong, Senzhang Wang, and Jianxin Wang. 2024. Fine Tuning Out-of-Vocabulary Item Recommendation with User Sequence Imagination. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[19] Taichi Liu, Chen Gao, Zhenyu Wang, Dong Li, Jianye Hao, Depeng Jin, and Yong Li. 2023. Uncertainty-aware Consistency Learning for Cold-Start Item Recommendation. In *Conference on Information Retrieval (SIGIR)*.

[20] Feiyang Pan, Shuokai Li, Xiang Ao, Pingzhong Tang, and Qing He. 2019. Warm up cold-start advertisements: Improving ctr predictions via learning to learn id embeddings. In *Conference on Information Retrieval (SIGIR)*.

[21] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Conference on Uncertainty in Artificial Intelligence (UAI)*.

[22] Scott Sanner, Krisztian Balog, Filip Radlinski, Ben Wedin, and Lucas Dixon. 2023. Large language models are competitive near cold-start recommenders for language-and item-based preferences. In *Conference on Recommender Systems (RecSys)*.

[23] Shaoyun Shi, Min Zhang, Xinxing Yu, Yongfeng Zhang, Bin Hao, Yiqun Liu, and Shaoping Ma. 2019. Adaptive feature sampling for recommendation with missing content feature values. In *International Conference on Information and Knowledge Management (CIKM)*.

[24] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research (JMLR)* (2014).

[25] Aaron Van den Oord, Sander Dieleman, and Benjamin Schrauwen. 2013. Deep content-based music recommendation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[26] Maksims Volkovs, Guangwei Yu, and Tomi Poutanen. 2017. Dropoutnet: Addressing cold start in recommender systems. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[27] Hao Wang, Binyi Chen, and Wu-Jun Li. 2013. Collaborative topic regression with social regularization for tag recommendation. In *Conference on Artificial Intelligence (AAAI)*.

[28] Jianling Wang, Haokai Lu, James Caverlee, Ed H Chi, and Minmin Chen. 2024. Large Language Models as Data Augmenters for Cold-Start Item Recommendation. In *International World Wide Web Conference (WWW)*.

[29] Wenbo Wang, Ben Chen, Bingquan Liu, Xinxin Wang, Luwei Yang, Wen Jiang, Wei Ning, and Jian Guan. 2024. Mutual Information Assisted Graph Convolution Network for Cold-Start Recommendation. In *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

[30] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Conference on Information Retrieval (SIGIR)*.

[31] Jian Wei, Jianhua He, Kai Chen, Yi Zhou, and Zuoyin Tang. 2017. Collaborative filtering and deep learning based recommendation system for cold start items. *Expert Systems with Applications* (2017).

[32] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2023. Llmrec: Large language models with graph augmentation for recommendation. *arXiv preprint arXiv:2311.00423* (2023).

[33] Yinwei Wei, Xiang Wang, Qi Li, Liqiang Nie, Yan Li, Xuanping Li, and Tat-Seng Chua. 2021. Contrastive learning for cold-start recommendation. In *International Conference on Multimedia (MM)*.

[34] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.

[35] Zhibo Xiao, Luwei Yang, Wen Jiang, Yi Wei, Yi Hu, and Hao Wang. 2020. Deep multi-interest network for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2265–2268.

[36] Xiaoxiao Xu, Chen Yang, Qian Yu, Zhiwei Fang, Jiaxing Wang, Chaosheng Fan, Yang He, Changping Peng, Zhangang Lin, and Jingping Shao. 2022. Alleviating Cold-start Problem in CTR Prediction with A Variational Embedding Learning Framework. In *International World Wide Web Conference (WWW)*.

[37] Jingkang Yang, Kaiyang Zhou, Yixuan Li, and Ziwei Liu. 2024. Generalized out-of-distribution detection: A survey. *International Journal of Computer Vision (IJCV)* (2024).

[38] Yijie Zhang, Yuanchen Bei, Hao Chen, Qijie Shen, Zheng Yuan, Huan Gong, Senzhang Wang, Feiran Huang, and Xiao Huang. 2024. Multi-behavior collaborative filtering with partial order graph convolutional networks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6257–6268.

[39] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2024. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering* (2024).

[40] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Conference on Artificial Intelligence (AAAI)*.

[41] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Conference on Knowledge Discovery and Data Mining (KDD)*.
[42] Yongchun Zhu, Kaikai Ge, Fuzhen Zhuang, Ruobing Xie, Dongbo Xi, Xu Zhang, Leyu Lin, and Qing He. 2021. Transfer-meta framework for cross-domain recommendation to cold-start users. In *Conference on Information Retrieval (SIGIR)*.
[43] Yongchun Zhu, Ruobing Xie, Fuzhen Zhuang, Kaikai Ge, Ying Sun, Xu Zhang, Leyu Lin, and Juan Cao. 2021. Learning to warm up cold item embeddings for cold-start recommendation with meta scaling and shifting networks. In *Conference on Information Retrieval (SIGIR)*.
[44] Ziwei Zhu, Jingu Kim, Trung Nguyen, Aish Fenton, and James Caverlee. 2021. Fairness among new items in cold start recommender systems. In *Conference on Information Retrieval (SIGIR)*.
[45] Ziwei Zhu, Shahin Sefati, Parsa Saadatpanah, and James Caverlee. 2020. Recommendation for new users and new items via randomized training and mixture-of-experts transformation. In *Conference on Information Retrieval (SIGIR)*.